Pixels Are (Probably) All You Need

Neil Jones

Final report for ECE 209-AS: Trustworthy AI
Apr 2026

Abstract

Large Vision Language Models (LVLMs) are prone to Object Hallucination (OH), in which the model describes objects that are not present, or refuses to recognize objects that are. An important class of techniques to mitigate OH is based around contrastive decoding, which perturbs either the text or image input and adjusts the output logits prior to sampling. In this work we demonstrate that, in fact, complex image perturbation methods are largely unnecessary in visual-based methods, while text-based contrastive decoding underperform. In other words, to perform successful contrastive decoding, one simply requires a blank image of the same size as the original.

1 Background

If $\mathcal{T}$ represents all text sequences and $\mathcal{I}$ represents all images, Large Vision Language Models (LVLMs [8]) describe a model architecture that provide a function $f : \mathcal{T} \times \mathcal{I} \to \mathcal{T}$ that can effectively "answer queries" about an image. One phenomenon exhibited by LVLMs is Object Hallucination (OH), where an LVLM's output may indicate the presence of an object that does not exist in the input image, or deny the presence of an object that does exist in the image. It is theorized [4] that one important cause of OH arises from statistical bias in language itself; that is, in LVLM training data, presumably there are more co-occurrences of "cat" and "dog" than there are with "cat" and "clown" so if a picture that involves a cat is provided with a prompt "Describe the animals in this image," an LVLM may be predisposed to claim the existence of both a cat and a dog simply because the cat's image embedding will trigger "dog" because of linguistic bias.

Techniques to reduce OH in LVLMs have considerable practical benefit and much work has been done on the problem in multiple areas [2]. One particularly important type of mitigation for OH involves so-called *training free* adjustments, which can be applied to any LVLM without fine-tuning. Training free adjustments allow a practitioner to correct a misbehaving model without altering the model parameters, usually by changing the autoregressive logic that is exercised during the inference phase. We draw a distinction here between "pure" training-free methods (eg, ICD and VCD) which rely only on the original image, prompt, and model, versus "assisted" training-free methods which rely on an assistant model or workflow to perform specific vision classification tasks by inspecting the image content itself (eg, [10]). In particular, we want to assess how far pure training-free methods can go in terms of mitigating OH.

One such training free method that has shown promise is Visual Contrastive Decoding (VCD [4]). In this method, the original text prompt p is presented to the LVLM along with the image I, and a modified image I' is used with the same prompt to generate a different output. At each step in the autoregressive loop, the logits of the output from the unmodified image query are adjusted by the logits of the modified image query with the intention of reducing the language bias. The original method described in [4] used a diffusion based noise generator, and we consider several variations on that below.

Preprint.

A key aspect of studying methods that reduce OH is the methodology of measuring OH. The Probing-Based Object Polling Evaluation (POPE [5]) benchmark converts a series of images into yes/no queries about object existence within the image, and is provided on the well-known MSCOCO [6] dataset. I'll remark that many works that target OH in LVLMs rely on the POPE benchmark, but often do not agree in their baseline measurements (see Table 1), indicating a need to characterize the performance of POPE itself independently of the OH mitigation approach.

Table 1: Reported POPE accuracy on regular decoding within LLaVa-1.5 across 5 studies. The difference between studies can be of the same magnitude as the improvement from an OH mitigation attempt.

Setting	Study	Baseline decoding accuracy
Popular	[4]	$81.88(\pm 0.48)$
	[10]	76.67
	[3]	79.98
	[7]	79.57
	[1]	70.27
Random	[4]	$83.29(\pm 0.35)$
	[10]	86.00
	[3]	84.77
	[7]	81.07
	[1]	72.20
Adversarial	[4]	$78.96(\pm 0.52)$
	[10]	73.33
	[3]	76.03
	[7]	74.50
	[1]	69.23

1.1 This work

We document three main results for this technical report. First, we recapitulate the results of the original work on VCD [4] on the POPE benchmark, along with a less-noised version. Second, we consider several alternate image filters – and ICD [9] – to compare against the diffusion-based generator. Finally, we examine the POPE benchmarking methodology to derive a threshold which can be used for determining whether or not a particular mitigation method has improved upon the baseline under that benchmark.

2 Methods

2.1 Visual Contrastive Decoding

VCD as described in [4] involves three components (the specific methodology of VCD is not redescribed here). (1) A diffusion-inspired noise generator $\mathcal{G} : \mathcal{I} \to \mathcal{I}$ converts an image $I \in \mathcal{I}$ to a noisy version $I' = \mathcal{G}(I)$ by applying pixelwise gaussian noise in small steps; both I and I' are used for inference along with the original text prompt p, forming pairs (p, I) and (p, I'). (2) Logits in the autoregressive step for $f(p, I)$ are adjusted with a simple additive weighting scheme with the logits for $f(p, I')$. Finally, (3) logits are filtered through "adaptive probability constraints" to ensure that tokens are not over-penalized by the additive scheme in (2).

Of particular note is the function $\mathcal{G}$: this is a function we are free to choose. Figure 1 demonstrates several choices for the noising function.

2.2 Instructive Contrastive Decoding

ICD as described in [9] involves the same basic process as VCD, except that the image is kept the same and the prompt is changed. The ICD paper describes two kinds of prompt perturbations, "positive" and "negative", which are exemplified by the prompt prefixes shown in 2. The remainder of the inference process is equivalent to VCD, including the adaptive probability constraints, but the original image (unmodified) is used.

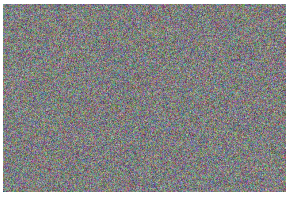
Figure 1: Different image perturbation techniques. For each filter shown, various parameters were tested that had different effects on the overall structure of the image.



(a) Original image



(b) All black



(c) Random



(d) VCD 100 steps



(e) VCD 250 steps



(f) VCD 500 steps (baseline VCD)



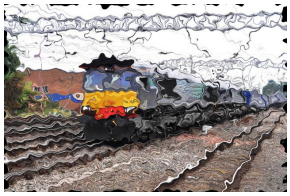
(g) Elastic ($\alpha = 100$)



(h) Elastic ($\alpha = 150$)



(i) Elastic ($\alpha = 200$)



(j) Elastic ($\alpha = 250$)



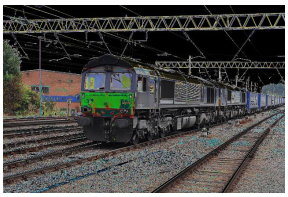
(k) Gauss (25, 35)



(l) Solarized ($t = 100$)



(m) Solarized ($t = 150$)



(n) Solarized ($t = 190$)

P1	You are an object detector to recognize every different object.
P2	You are an object detector to recognize every different object by focusing on the shapes, colors, and relationships of objects.
P3	You are an incredibly powerful object detector that rarely makes mistakes because of how much attention you pay to all of the attributes of an image.
N1	I want you to avoid any specific identification or categorization of the objects depicted.
N2	You are a confused object detector to provide a fuzzy overview or impression of the image.
N3	You are an object detector and you are incompetent at your job. You are hardly ever right, but you have to try anyway.

Table 2: Prompt suffixes used in ICD tests. These are, for the most part, taken from the original paper.

2.3 Implementation Notes

The experiments performed in this work were performed on a newer GPU (nvidia RTX-5070) than used in [4]. This required two main departures from that paper: first, the LLaVa-1.5 model was officially adopted by the HuggingFace Transformers library after the publication of the original paper, and differs slightly from the original LLaVa-1.5 model. Second, the version of HuggingFace Transformers and pytorch necessary to run the model required a newer version of CUDA, and in the process, negated much of the code written by the original VCD authors which relied on overriding deprecated and private methods. As such, it was necessary to reimplement the components of VCD in a way that is compatible with the current Transformers Library.

Another departure from the original paper is in the decoding process. Rather than creating a single input batch with n text prompts and images, we create an input batch with $2n$ text prompts, two for each image: one original and one noised. The original decoding process described in [4] performed two forward passes for every token extension, and our implementation performs only one. The logits for sampling are adjusted in a custom `LogitsProcessor` which performs the normalization and filtering for parts (2) and (3) above. However, one problem remains, which is the actual sampling of the token. It is important during sampling that the context window used with both images retain the same token history – and for this, a custom `StoppingCriteria` can simply overwrite the sampled token for the noised image with the sampled token from the non-noised image, preserving the required relationship for the next sampling step. Though inelegant, this implementation approach uses the public interfaces for the Transformers library and should be more resilient to future internal library changes. One drawback to this method is that it uses twice the GPU memory for a given batch size.

Finally, the LLaVa-1.5 13B model was run in 4-bit quantized mode, and Qwen3-VL-Instruct was run in 8-bit quantized mode; LLaVa-1.5 7B was run in full precision.

2.4 POPE

The POPE benchmark [5] consists of a series of yes/no object existence questions paired to an image based on a golden set. As mentioned above, I found disagreement within the existing literature for the results of POPE run on an unmodified LLaVa-1.5 model, suggesting one or more of: (a) methodological differences in executing POPE; (b) random variance among equivalent POPE executions; (c) implementation errors or data set differences. I also observed several edge cases with POPE in our local evaluation. For instance, the LVLM under study would sometimes return more accurate, and comical, one-word answers that were not yes or no: "Is there a bird in this image? Answer in one word." would, with some regularity, return with "Chicken". And indeed, there were chickens in the corresponding photo. In other cases, the model would respond with "Nope" or "Yeah". If the prompt changed to "Is there a bird in this image? Answer in one word, **yes or no**." results changed slightly.

This raises an issue of whether a particular OH mitigation method's improvements are significant or not: one cannot assume the POPE benchmark itself is without variance. What justification do we have that the particular formulation of the POPE benchmark used in a particular paper is not just more amenable to that particular mitigation method? To test this we created a list of 12 prompt suffixes that were semantically equivalent (see Table 3) to the original POPE prompts and re-evaluated the

baseline inference (with no model modifications) to determine the sensitivity of POPE to the format of the prompt.

Suppose the original POPE prompt format is p_0, and we have m additional prompt formats p_i that are all semantically equivalent for $i \in \{1, 2, \ldots, m\}$. If we take one particular metric, say Accuracy, and compute s_i^2 as the sample variance of that metric over n runs of POPE with prompt p_i, then the standard error of the difference of the Accuracy between p_0 and p_i is $SE_i = \sqrt{\frac{s_i^2 + s_0^2}{n}}$. We can then compute a threshold value $T_i = t_{\alpha,df} SE_i$, where $t_{\alpha,df}$ is the α-quantile of the T-distribution with df degrees of freedom (computed via the one-sided (greater) Welch t-test for independent samples with possibly unequal variance). Over the m alternate prompts, we locate the largest T_i, and declare this (empirically) to be τ_n, the floor of significance over n samples: any OH mitigation treatment that shows less than τ_n improvement over the baseline decoding cannot claim to be significantly better than the baseline given the random variations in the benchmark itself. This essentially uses a repeated Welch t-test to estimate a critical value in order to avoid having to repeat this analysis in every benchmarking study.

Table 3: Different prompts used to test the sensitivity of the POPE benchmark on prompt format. Each prompt is semantically equivalent, but preserves the need for a yes or no response.

Prompt id	POPE Suffix text
default	Please answer this question with one word.
default2	Please answer this question with a single word.
yesno	Please answer this question with one word, “yes” or “no”.
yesno2	Please answer this question with a single word, “yes” or “no”.
rude	Answer this question with a only one word.
rude2	Answer this question with a only a single word.
rudeyesno	Answer this question with one word, “yes” or “no”.
rudeyesno2	Answer this question with a single word, “yes” or “no”.
brief	“Yes” or “no”, one word.
brief2	“Yes” or “no”, a single word.
long	Your answer should contain only one word, “yes” or “no”.
long2	Your answer should contain only a single word, “yes” or “no”.
none2	Yes or no?

3 Results

3.1 POPE VCD

While the experimental results in Table 4 are surprisingly similar to the original paper, and generally closer than any of the other studies shown in Table 1, they are not equivalent to within rounding error. This may stem from (a) differences in implementing the POPE benchmark, particularly around answer handling; (b) differences in model weights, because the LLaVa model used in this study was slightly different as noted above; (c) underlying software version differences; (d) subtle differences in the implementations; (e) quantization differences between the models: I have no idea what sized models the original paper used nor their quantization settings. Notably, however, VCD on Qwen3-VL proved to have either no real effect or a slightly negative one, which is a departure from the original paper. Note that Qwen3 post-dates the original paper and generally performs better on POPE than the version listed, which was presumably Qwen2; Qwen3’s performance was near perfect on the benchmark, so the problem of OH in that model is largely moot as far as this particular benchmark is concerned. Obviously one would be foolish to claim that Qwen3 has solved object hallucination in general, but instead should conclude that there is a need to develop stronger benchmarks for future models. As expected we tend to see higher performance with more sophisticated models, and there is not much headroom remaining so POPE as described may well lose its discriminative value as the power of the models increase.

Table 4: Evaluation of POPE on regular (baseline) and VCD decoding with three models. This is a recapitulation of the same table in [4], but includes additional image noisers. In nearly all cases, the accuracy increased within a given image noiser family, as the amount of noise was increased; accordingly, we show only the highest setting tested. There were no significant differences between prompt prefixes for ICD, so only one version is shown.

Setting	Model	Decoding	Accuracy	Precision	Recall	F1 Score
adversarial	llava-13b	baseline	79.56(±0.64)	84.48(±1.08)	72.44(±0.74)	77.99(±0.65)
		vcd	82.06(±0.36)	79.86(±0.35)	85.75(±0.43)	82.70(±0.35)
	llava-7b	baseline	78.21(±0.50)	81.82(±0.75)	72.53(±0.63)	76.90(±0.51)
		black	78.99(±0.31)	73.75(±0.27)	90.00(±0.29)	81.07(±0.27)
		ela_250	80.25(±0.42)	81.57(±0.55)	78.15(±0.45)	79.82(±0.42)
		gauss	79.69(±0.37)	80.81(±0.45)	77.88(±0.28)	79.32(±0.35)
		icd_n1	79.47(±0.47)	84.67(±0.51)	71.96(±0.66)	77.80(±0.53)
		rand	78.97(±0.28)	74.09(±0.30)	89.11(±0.23)	80.91(±0.23)
		sol_190	79.91(±0.30)	83.70(±0.39)	74.28(±0.61)	78.71(±0.37)
		vcd	79.11(±0.36)	74.29(±0.33)	89.04(±0.33)	81.00(±0.31)
		vcd_n250	80.12(±0.41)	81.38(±0.41)	78.11(±0.51)	79.71(±0.43)
	qwen3-8b	baseline	84.92(±0.10)	87.47(±0.18)	81.52(±0.13)	84.39(±0.10)
		black	84.30(±0.00)	84.28(±0.00)	84.33(±0.00)	84.31(±0.00)
		ela_250	84.21(±0.28)	84.61(±0.28)	83.63(±0.35)	84.11(±0.29)
		gauss	84.36(±0.20)	85.32(±0.22)	83.00(±0.19)	84.14(±0.20)
		rand	83.60(±0.00)	83.42(±0.00)	83.87(±0.00)	83.64(±0.00)
		sol_190	84.49(±0.18)	86.49(±0.27)	81.76(±0.15)	84.06(±0.17)
		vcd	84.31(±0.15)	84.30(±0.12)	84.32(±0.24)	84.31(±0.16)
		vcd_n250	84.38(±0.09)	85.24(±0.23)	83.16(±0.17)	84.19(±0.08)
popular	llava-13b	baseline	81.85(±0.49)	89.61(±0.91)	72.07(±0.23)	79.89(±0.49)
		vcd	86.11(±0.26)	86.33(±0.25)	85.80(±0.43)	86.06(±0.28)
	llava-7b	baseline	81.01(±0.72)	87.33(±1.33)	72.57(±0.68)	79.26(±0.71)
		black	85.13(±0.37)	81.99(±0.41)	90.04(±0.30)	85.83(±0.34)
		ela_250	83.21(±0.39)	86.93(±0.41)	78.16(±0.44)	82.31(±0.42)
		gauss	83.01(±0.39)	86.78(±0.63)	77.88(±0.26)	82.09(±0.37)
		icd_n1	82.10(±0.49)	90.25(±0.54)	71.97(±0.66)	80.08(±0.58)
		rand	84.72(±0.25)	82.00(±0.35)	88.97(±0.29)	85.34(±0.22)
		sol_190	82.89(±0.36)	89.70(±0.45)	74.32(±0.60)	81.29(±0.43)
		vcd	84.83(±0.33)	82.11(±0.36)	89.05(±0.31)	85.44(±0.30)
		vcd_n250	83.35(±0.40)	87.33(±0.51)	78.01(±0.46)	82.41(±0.43)
	qwen3-8b	baseline	86.60(±0.12)	90.74(±0.19)	81.53(±0.15)	85.89(±0.13)
		black	87.10(±0.00)	89.05(±0.00)	84.60(±0.00)	86.77(±0.00)
		ela_250	86.51(±0.31)	88.64(±0.36)	83.75(±0.39)	86.12(±0.32)
		gauss	86.45(±0.13)	89.09(±0.27)	83.08(±0.20)	85.98(±0.13)
		rand	86.77(±0.00)	88.70(±0.00)	84.27(±0.00)	86.43(±0.00)
		sol_190	86.59(±0.18)	90.45(±0.32)	81.83(±0.12)	85.92(±0.17)
		vcd	86.67(±0.09)	88.36(±0.13)	84.48(±0.21)	86.37(±0.10)
		vcd_n250	86.84(±0.09)	89.65(±0.12)	83.29(±0.26)	86.36(±0.11)
random	llava-13b	baseline	82.86(±0.47)	91.53(±0.61)	72.43(±0.74)	80.86(±0.56)
		vcd	89.10(±0.33)	91.86(±0.46)	85.80(±0.43)	88.73(±0.34)
	llava-7b	baseline	82.47(±0.48)	90.50(±0.95)	72.57(±0.68)	80.55(±0.51)
		black	88.33(±0.17)	87.07(±0.16)	90.04(±0.30)	88.53(±0.18)
		ela_250	85.59(±0.26)	91.80(±0.54)	78.16(±0.44)	84.43(±0.27)
		gauss	85.07(±0.21)	90.97(±0.36)	77.88(±0.26)	83.92(±0.22)
		icd_n1	83.53(±0.27)	93.60(±0.42)	71.97(±0.66)	81.37(±0.36)
		rand	88.44(±0.19)	88.06(±0.26)	88.95(±0.28)	88.50(±0.19)
		sol_190	84.43(±0.29)	93.17(±0.31)	74.32(±0.60)	82.68(±0.37)
		vcd	88.46(±0.21)	88.01(±0.23)	89.05(±0.31)	88.53(±0.21)
		vcd_n250	85.49(±0.30)	91.63(±0.38)	78.12(±0.51)	84.34(±0.35)
	qwen3-8b	baseline	89.88(±0.04)	97.87(±0.09)	81.53(±0.15)	88.96(±0.05)
		black	90.93(±0.00)	96.87(±0.00)	84.60(±0.00)	90.32(±0.00)
		ela_250	90.62(±0.25)	97.09(±0.16)	83.75(±0.39)	89.93(±0.28)
		gauss	90.23(±0.15)	96.94(±0.13)	83.08(±0.20)	89.47(±0.16)
		rand	90.93(±0.00)	96.80(±0.00)	84.67(±0.00)	90.33(±0.00)
		sol_190	89.90(±0.12)	97.58(±0.15)	81.83(±0.12)	89.01(±0.13)
		vcd	90.85(±0.12)	96.82(±0.16)	84.48(±0.21)	90.23(±0.13)
		vcd_n250	90.37(±0.11)	97.03(±0.14)	83.29(±0.26)	89.64(±0.13)

3.2 POPE baseline

The results of the POPE baseline study are shown in Table 4. There is general agreement among the different prompts, and considering the 10 different runs across the prompt options we obtain $\tau_{10} = 0.56$. However, the threshold increases as n decreases. Notably, the benchmarks in several studies were performed with 5 duplicate runs, and repeating our analysis we obtain $\tau_5 = 1.13$; this places the improvements for VCD on LLaVa-1.5-13b on the borderline of significance for the Adversarial split, and we see that Qwen3 is already indistinguishable from noise.

Table 5: Results of regular decoding for POPE, using semantically equivalent prompts. Collected on 10 runs each.

Prompt	n	Accuracy	T_m
Baseline	10	$81.15(\pm 0.61)$	
brief	10	$77.07(\pm 0.67)$	0.50
brief2	10	$76.73(\pm 0.62)$	0.48
default2	10	$81.54(\pm 0.62)$	0.48
long	10	$77.68(\pm 0.58)$	0.46
long2	10	$78.06(\pm 0.83)$	0.56
rude	10	$81.13(\pm 0.61)$	0.47
rude2	10	$81.68(\pm 0.62)$	0.47
rudeyesno	10	$79.77(\pm 0.53)$	0.44
yesno	10	$79.85(\pm 0.49)$	0.43
yesno2	10	$79.83(\pm 0.61)$	0.47

4 Conclusion and Further Work

It is notable that when using the diffusion noise generator, the contrastive decoding accuracy was reliably less when using 250 steps instead of 500; as figure 1 shows, 500 steps obliterates any visual signal in the image. Oddly, an image of completely random pixels underperforms the diffusion noise with 500 steps. However, an image with blank pixels beats all other methods, generally by a statistically significant amount.

Contrary to claims in [9], we did not find instructional contrastive decoding to have a significant improvement over vanilla VCD, and often it underperformed. It is possible that alternate prompting strategies could debias the linguistic priors, but combined with the observation that a blank image obtained the best improvement, it appears that the role of the image is paramount in contrastive effect, but absurdly, the contents of the pixels is arbitrary: they simply must be as different as possible from the original image.

One key limitation of this study is the quality of LVLMs compared to the difficulty of the benchmark. Despite minor improvements in one or the other strategies, the models used scored high enough on the benchmark to imply that the benchmark does not have sufficient discriminative power. As such, this work suggests a possible conclusion about the technique of Visual Contrastive Decoding, but requires additional work with other benchmarks such as CHAIR, and ideally the development of new benchmarking methods for the task of OH mitigation, in order to be definitive.

5 References

References

[1] Z. Chen, Z. Zhao, H. Luo, H. Yao, B. Li, and J. Zhou. HALC: Object Hallucination Reduction via Adaptive Focal-Contrast Decoding, June 2024. URL `http://arxiv.org/abs/2403.00425`. arXiv:2403.00425.

[2] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems*, 43(2):1–55, Mar. 2025. ISSN 1046-8188, 1558-2868. doi: 10.1145/3703155. URL `https://dl.acm.org/doi/10.1145/3703155`.

[3] F. Huo, W. Xu, Z. Zhang, H. Wang, Z. Chen, and P. Zhao. Self-Introspective Decoding: Alleviating Hallucinations for Large Vision-Language Models, Mar. 2025. URL http://arxiv.org/abs/2408.02032. arXiv:2408.02032.

[4] S. Leng, H. Zhang, G. Chen, X. Li, S. Lu, C. Miao, and L. Bing. Mitigating Object Hallucinations in Large Vision-Language Models through Visual Contrastive Decoding, Nov. 2023. URL http://arxiv.org/abs/2311.16922. arXiv:2311.16922 [cs].

[5] Y. Li, Y. Du, K. Zhou, J. Wang, W. X. Zhao, and J.-R. Wen. Evaluating Object Hallucination in Large Vision-Language Models, Oct. 2023. URL http://arxiv.org/abs/2305.10355. arXiv:2305.10355 [cs].

[6] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár. Microsoft COCO: Common Objects in Context, Feb. 2015. URL http://arxiv.org/abs/1405.0312. arXiv:1405.0312 [cs].

[7] B. Liu, F. Zhang, G. Chen, and J. Cheng. Multifrequency Contrastive Decoding: Alleviating Hallucinations for Large Vision-Language Models. *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Nov. 2025. URL https://aclanthology.org/2025.emnlp-main.1452.pdf.

[8] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning Transferable Visual Models From Natural Language Supervision, Feb. 2021. URL http://arxiv.org/abs/2103.00020. arXiv:2103.00020 [cs].

[9] X. Wang, J. Pan, L. Ding, and C. Biemann. Mitigating Hallucinations in Large Vision-Language Models with Instruction Contrastive Decoding, June 2024. URL http://arxiv.org/abs/2403.18715. arXiv:2403.18715.

[10] S. Yin, C. Fu, S. Zhao, T. Xu, H. Wang, D. Sui, Y. Shen, K. Li, X. Sun, and E. Chen. Woodpecker: hallucination correction for multimodal large language models. *Science China Information Sciences*, 67(12):220105, Dec. 2024. ISSN 1674-733X, 1869-1919. doi: 10.1007/s11432-024-4251-x. URL https://link.springer.com/10.1007/s11432-024-4251-x.

6 Code

Following are the main code modules used in this work.

6.1 Implementation of VCD

This file contains the actual contrastive decoding logic, but not the plumbing.

Listing 1: vcd.py

```python
"""Simplified version of VCD to assist with benchmarking."""


import torch
from torch import nn
from transformers import LogitsProcessor, StoppingCriteria


class VCDLogitsProcessor(LogitsProcessor):
    # These are the values described in the paper, but the code in the github repo actually defaults to
    # to 0.5 for alpha.
    def __init__(self, alpha=1.0, beta=0.1):
        super().__init__()  # Chain, just in case.
        self._alpha = alpha
        self._beta = beta

    """This applies the VCD rule to the logits."""
```

```
    def __call__(self, input_ids: torch.LongTensor, scores: torch.FloatTensor) -> torch.FloatTensor:
        """Note: thsi does not apply adaptive plausability constraitns."""
        alpha = self._alpha
        beta = self._beta
        max_idx = int(input_ids.shape[0] / 2)

        # Compute the cutoff for APC, according to what is in the paper (rather than what is in the code)
        # (Beta * max prob). This means we have to convert logits into probability space...
        probs = nn.functional.softmax(scores, dim=-1)
        # ...and compute that threshold; note this will include thresholds for the noised tensors as well,
        # but it doesn't matter because we overwrite their sampling later.
        cutoff = beta * probs.max(dim=-1, keepdim=True).values

        for i in range(max_idx):
            scores[2*i] = (1 + alpha) * scores[2*i] - alpha * scores[2*i+1]

        # Apply APC. Using -infinity makes the probability 0.
        scores = scores.masked_fill(probs < cutoff, -float("inf"))

        return scores


class TokenReplacementHook(StoppingCriteria):
    """This is a hack to allow the tokens to be properly sync'ed between the original and noised image."""
    def __call__(self, input_ids, scores, **kwargs):
        max_idx = int(input_ids.shape[0] / 2)
        if input_ids.shape[0] % 2 == 0:
            for i in range(max_idx):
                input_ids[2*i + 1, -1] = input_ids[2*i, -1]
        # Never want to stop early, so this stopping criteria is always false.
        return False


# I lifted this from the original VCD code. It looks familiar to what I wrote in ECE239AS but is
# a bit tighter. I like it!
def add_diffusion_noise(image_tensor, noise_step):
    """Given an image (h, w, c), run gaussian diffusion for a set number of steps."""

    num_steps = 1000  # Number of diffusion steps

    # decide beta in each step
    betas = torch.linspace(-6,6,num_steps)
    betas = torch.sigmoid(betas) * (0.5e-2 - 1e-5) + 1e-5

    # decide alphas in each step
    alphas = 1 - betas
    alphas_prod = torch.cumprod(alphas, dim=0)
    alphas_prod_p = torch.cat([torch.tensor([1]).float(), alphas_prod[:-1]],0) # p for previous
    alphas_bar_sqrt = torch.sqrt(alphas_prod)
    one_minus_alphas_bar_log = torch.log(1 - alphas_prod)
    one_minus_alphas_bar_sqrt = torch.sqrt(1 - alphas_prod)

    def q_x(x_0,t):
```

```
        noise = torch.randn_like(x_0)
        alphas_t = alphas_bar_sqrt[t]
        alphas_1_m_t = one_minus_alphas_bar_sqrt[t]
        return (alphas_t*x_0 + alphas_1_m_t*noise)

    noise_delta = int(noise_step) # from 0-999
    noisy_image = image_tensor.clone()
    image_tensor_cd = q_x(noisy_image,noise_step)

    return image_tensor_cd
```

6.2 Model loading and basic benchmark code

This file defines which models are available for benchmarking and how to load them on my particular hardware.

Listing 2: configs.py

```
import logging
import benchmark
import torch
from transformers import AutoProcessor, BitsAndBytesConfig,
    InstructBlipForConditionalGeneration,
    LlavaForConditionalGeneration, Qwen3VLForConditionalGeneration,
    set_seed

MODELS = {
        "llava-7b": {
            "MODEL_ID": "llava-hf/llava-1.5-7b-hf",
            "BIG_FAT_MODEL": False,
            "GEN": LlavaForConditionalGeneration,
        },
        "llava-13b": {
            "MODEL_ID": "llava-hf/llava-1.5-13b-hf",
            "REALLY_BIG_FAT_MODEL": True,
            "GEN": LlavaForConditionalGeneration,
        },
        "qwen3-8b": {
            "MODEL_ID": "Qwen/Qwen3-VL-8b-Instruct",
            "BIG_FAT_MODEL": True,
            "GEN": Qwen3VLForConditionalGeneration,
        },
        # NOTE: instructblip is not yet supported.
        "instructblip-7b": {
            "MODEL_ID": "Salesforce/instructblip-vicuna-7b",
            "BIG_FAT_MODEL": True,
            "GEN": InstructBlipForConditionalGeneration,
        },
        "instructblip-13b": {
            "MODEL_ID": "Salesforce/instructblip-vicuna-13b",
            "REALLY_BIG_FAT_MODEL": True,
            "GEN": InstructBlipForConditionalGeneration,
        },
}


def load_model(name: str):
    """Returns the model to be used in the benchmarking test."""
    logging.warning("Loading model")
    # Use the officially supported LLAVA model in HuggingFace
    if name not in MODELS:
        raise Exception(f"Unknown model {name}")

    if MODELS[name].get("REALLY_BIG_FAT_MODEL", False):
```

```
        bnb_conf = BitsAndBytesConfig(load_in_4bit=True)
        model = MODELS[name]["GEN"].from_pretrained(
            MODELS[name]["MODEL_ID"], dtype=torch.float16,
                low_cpu_mem_usage=False, quantization_config=bnb_conf)
    elif MODELS[name].get("BIG_FAT_MODEL", False):
        bnb_conf = BitsAndBytesConfig(load_in_8bit=True)
        model = MODELS[name]["GEN"].from_pretrained(
            MODELS[name]["MODEL_ID"], dtype=torch.float16,
                low_cpu_mem_usage=False, quantization_config=bnb_conf)
    else:
        model = MODELS[name]["GEN"].from_pretrained(
            MODELS[name]["MODEL_ID"], dtype=torch.float16,
                low_cpu_mem_usage=False).to(0)
    return model


def load_processor(name: str):
    """Returns the correct (input) processor to use for the model."""
    logging.warning("Loading processor")
    return AutoProcessor.from_pretrained(MODELS[name]["MODEL_ID"])


def load_benchmark(kind: str, processor) -> benchmark.Benchmark:
    return benchmark.REGISTRY[kind](processor)


def query_model(model, benchmark, processor, prompt, image,
    max_new_tokens=20):
    inputs = benchmark.prepare(prompt, image)
    inputs["max_new_tokens"] = max_new_tokens
    inputs["do_sample"] =  True

    custom_logits_processor = benchmark.logits_processor()
    if custom_logits_processor:
        inputs["logits_processor"] = [custom_logits_processor]

    stopping_criteria = benchmark.stopping_criteria()
    if stopping_criteria:
        inputs["stopping_criteria"] = [stopping_criteria]

    output = model.generate(**inputs)
    response = processor.decode(output[0], skip_special_tokens=True)
    return response
```

This file defines the main interfaces that will define how benchmarking works.

Listing 3: benchmark.py

```
"""A module to facilitate benchmark studies."""

from abc import abstractmethod, ABC
from PIL import Image
import numpy as np
import torch
from torchvision.transforms import transforms
from torchvision.transforms import v2
import vcd


class Benchmark(ABC):
    def __init__(self, processor=None):
        self.processor = processor

    @abstractmethod
    def prepare(self, question: str, image: Image):
```

```
        """Takes a question prompt and an image and returns a tensor or something."""
        ...

    @abstractmethod
    def logits_processor(self):
        """Return a custom LogitsProcessor, as necessary; return None if no special processing is done."""
        ...

    @abstractmethod
    def stopping_criteria(self):
        ...


class Baseline(Benchmark):
    """The baseline is a straight eval of LLava 1.5 using POPE."""
    def prepare(self, question, image: Image):
        return self.processor(images=[image], text=[question], return_tensors='pt').to(0, torch.float16)

    def logits_processor(self):
        return None

    def stopping_criteria(self):
        return None


PIL_TO_TENSOR = transforms.ToTensor()
TENSOR_TO_PIL = transforms.ToPILImage()

class BaseICDBenchmark(Benchmark):
    def __init__(self, processor):
        super().__init__(processor)
        self.custom_logits_processor = vcd.VCDLogitsProcessor()
        self.custom_stopping_criteria = vcd.TokenReplacementHook()

    def prepare(self, question: str, image: Image):
        disturbed_question = self.disturb(question)
        return self.processor(images=[image, image], text=[question, disturbed_question], padding=True, return_tensors='pt').to(0, torch.float16)

    def logits_processor(self):
        return self.custom_logits_processor

    def stopping_criteria(self):
        return self.custom_stopping_criteria

    @abstractmethod
    def disturb(self, question: str) -> str:
        ...


class P1ICDBenchmark(BaseICDBenchmark):
    def disturb(self, question: str) -> str:
        return 'You are an object detector to recognize every different object. ' + question


class P2ICDBenchmark(BaseICDBenchmark):
    def disturb(self, question: str) -> str:
        return 'You are an object detector to recognize every different object by focusing on the shapes, colors, and relationships of objects. ' + question
```

```python


class P3ICDBenchmark(BaseICDBenchmark):
    def disturb(self, question: str) -> str:
        return 'You are an incredibly powerful object detector that rarely makes mistakes because of how much attention you pay to all of the attributes of an image. ' + question


class N1ICDBenchmark(BaseICDBenchmark):
    def disturb(self, question: str) -> str:
        return 'I want you to avoid any specific identification or categorization of the objects depicted. ' + question


class N2ICDBenchmark(BaseICDBenchmark):
    def disturb(self, question: str) -> str:
        return 'You are a confused object detector to provide a fuzzy overview or impression of the image. ' + question


class N3ICDBenchmark(BaseICDBenchmark):
    def disturb(self, question: str) -> str:
        return 'You are an object detector and you are incompetent at your job. You are hardly ever right, but you have to try anyway. ' + question


class VCDBenchmark(Benchmark):
    """This is a naive implementation of VCD suitable for POPE on Llava."""
    def __init__(self, processor):
        super().__init__(processor)
        self.custom_logits_processor = vcd.VCDLogitsProcessor()
        self.custom_stopping_criteria = vcd.TokenReplacementHook()

    def prepare(self, question: str, image: Image):
        noised_image = TENSOR_TO_PIL(vcd.add_diffusion_noise(PIL_TO_TENSOR(image), 500))
        return self.processor(images=[image, noised_image], text=[question, question], return_tensors='pt').to(0, torch.float16)

    def logits_processor(self):
        return self.custom_logits_processor

    def stopping_criteria(self):
        return self.custom_stopping_criteria


class VCDBenchmarkA0p5(VCDBenchmark):
    """Sets alpha=0. All the time and none of the benefit, at twice the price!"""
    def __init__(self, processor):
        super().__init__(processor)
        self.custom_logits_processor = vcd.VCDLogitsProcessor(alpha=0.5)
        self.custom_stopping_criteria = vcd.TokenReplacementHook()


class VCDBenchmarkA2(VCDBenchmark):
    """Sets alpha=2"""
    def __init__(self, processor):
        super().__init__(processor)
        self.custom_logits_processor = vcd.VCDLogitsProcessor(alpha=2)
```

```
        self.custom_stopping_criteria = vcd.TokenReplacementHook()


class VCDBenchmarkN1(VCDBenchmark):
    """Infinitesimal noise."""

    def prepare(self, question: str, image: Image):
        noised_image = TENSOR_TO_PIL(vcd.add_diffusion_noise(
            PIL_TO_TENSOR(image), 1))
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


class VCDBenchmarkN999(VCDBenchmark):
    """Infinitesimal noise."""

    def prepare(self, question: str, image: Image):
        noised_image = TENSOR_TO_PIL(vcd.add_diffusion_noise(
            PIL_TO_TENSOR(image), 999))
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


class VCDBenchmarkN250(VCDBenchmark):
    """Infinitesimal noise."""

    def prepare(self, question: str, image: Image):
        noised_image = TENSOR_TO_PIL(vcd.add_diffusion_noise(
            PIL_TO_TENSOR(image), 250))
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


class BlackBenchmark(VCDBenchmark):
    img = Image.new('RGB', (640,428))
    def prepare(self, question: str, image: Image):
        noised_image = self.img
        return self.processor(images=[image, noised_image], text=[
            question, question], padding=True, padding_side='left',
            return_tensors='pt').to(0, torch.float16)


class RandomBenchmark(VCDBenchmark):
    img = Image.fromarray(np.random.randint(low=0, high=255, size
        =(640, 428, 3), dtype=np.uint8))

    def prepare(self, question: str, image: Image):
        noised_image = self.img
        return self.processor(images=[image, noised_image], text=[
            question, question], padding=True, padding_side='left',
            return_tensors='pt').to(0, torch.float16)


class GaussBlurBenchmark(VCDBenchmark):
    blurrer = v2.GaussianBlur(kernel_size=(25, 35), sigma=(3, 9.))
    def prepare(self, question: str, image: Image):
        noised_image = self.blurrer(image)
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)
```

```
class Solarizer100Benchmark(VCDBenchmark):
    blurrer = v2.RandomSolarize(threshold=100)
    def prepare(self, question: str, image: Image):
        noised_image = self.blurrer(image)
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


class Solarizer150Benchmark(VCDBenchmark):
    blurrer = v2.RandomSolarize(threshold=150)
    def prepare(self, question: str, image: Image):
        noised_image = self.blurrer(image)
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


class Solarizer190Benchmark(VCDBenchmark):
    blurrer = v2.RandomSolarize(threshold=190)
    def prepare(self, question: str, image: Image):
        noised_image = self.blurrer(image)
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


class Elastic100Benchmark(VCDBenchmark):
    blurrer = v2.ElasticTransform(alpha=100)
    def prepare(self, question: str, image: Image):
        noised_image = self.blurrer(image)
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


class Elastic150Benchmark(VCDBenchmark):
    blurrer = v2.ElasticTransform(alpha=150)
    def prepare(self, question: str, image: Image):
        noised_image = self.blurrer(image)
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


class Elastic200Benchmark(VCDBenchmark):
    blurrer = v2.ElasticTransform(alpha=200)
    def prepare(self, question: str, image: Image):
        noised_image = self.blurrer(image)
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


class Elastic250Benchmark(VCDBenchmark):
    blurrer = v2.ElasticTransform(alpha=250)
    def prepare(self, question: str, image: Image):
        noised_image = self.blurrer(image)
        return self.processor(images=[image, noised_image], text=[
            question, question], return_tensors='pt').to(0, torch.
            float16)


REGISTRY = {
    'baseline': Baseline,
```

```
    'vcd': VCDBenchmark,
    'vcd_a0.5': VCDBenchmarkA0p5,
    'vcd_n1': VCDBenchmarkN1,
    'vcd_n999': VCDBenchmarkN999,
    'vcd_n250': VCDBenchmarkN250,
    'vcd_a2': VCDBenchmarkA2,
    'icd_p1': P1ICDBenchmark,
    'icd_p2': P2ICDBenchmark,
    'icd_p3': P3ICDBenchmark,
    'icd_n1': N1ICDBenchmark,
    'icd_n2': N2ICDBenchmark,
    'icd_n3': N3ICDBenchmark,
    'ela_100': Elastic100Benchmark,
    'ela_150': Elastic150Benchmark,
    'ela_200': Elastic200Benchmark,
    'ela_250': Elastic250Benchmark,
    'gauss': GaussBlurBenchmark,
    'sol_100': Solarizer100Benchmark,
    'sol_150': Solarizer150Benchmark,
    'sol_190': Solarizer190Benchmark,
    'black': BlackBenchmark,
    'rand': RandomBenchmark,
}
```

6.3 Driver programs

This file executes the actual benchmark.

Listing 4: run_benchmark.py

```
"""This is a driver program for running an evaluation of POPE for a
    particular benchmark."""

import argparse
import configs
import os
import re
import json
import benchmark
import torch
import logging
from tqdm import tqdm
import sys

from PIL import Image
from transformers import set_seed


class Accumulator(object):
    def __init__(self):
        self.tp = 0
        self.fp = 0
        self.tn = 0
        self.fn = 0
        self.n = 0
        self.errors = []

    def __str__(self):
        acc = (self.tp + self.tn) / self.n * 100
        return ",".join([str(self.tp), str(self.tn), str(self.fp), str
            (self.fn), f"{acc:.2f}%", str(self.errors)])


IMAGE_PATH = "/home/ncjones/.cache/lavis/coco/images/val2014"
```

```python
PROMPT_SUFFIXES = {
        'default': ' Please answer this question with one word.',
        'default2': ' Please answer this question with a single word.',
        'yesno': ' Please answer this question with one word, "yes" or "no".',
        'yesno2': ' Please answer this question with a single word, "yes" or "no".',
        'rude': ' Answer this question with a only one word.',
        'rude2': ' Answer this question with a only a single word.',
        'rudeyesno': ' Answer this question with one word, "yes" or "no".',
        'rudeyesno2': ' Answer this question with a single word, "yes" or "no".',
        'brief': ' "Yes" or "no", one word.',
        'brief2': ' "Yes" or "no", a single word.',
        'long': ' Your answer should contain only one word, "yes" or "no".',
        'long2': ' Your answer should contain only a single word, "yes" or "no".',
        'none': ' ',
        'none2': ' Yes or no?',
}

def load_questions(path: str, processor, max_num_questions, prompt_suffix='default'):
    """Read and process the POPE questions."""
    doubled = False
    tripled = False
    quadrupled = False
    if ':' in prompt_suffix:
        if prompt_suffix[0:8] == 'doubled:':
            doubled = True
        elif prompt_suffix[0:8] == 'tripled:':
            tripled = True
        elif prompt_suffix[0:8] == 'quadgod:':
            quadrupled = True
        else:
            raise Exception(f'Unknown prompt suffix {prompt_suffix}')
        prompt_suffix = prompt_suffix[8:]

    suffix = PROMPT_SUFFIXES.get(prompt_suffix, "")
    logging.warning(f"Reading in POPE: {suffix=}")
    questions = [json.loads(q) for q in open(os.path.expanduser(path), "r")]
    inputs = []
    for question in questions:
        prompt_text = question["text"] + " " + suffix
        # I can't even believe it's worth testing this..but arxiv never lies.
        if doubled:
            prompt_text += prompt_text
        elif tripled:
            prompt_text += prompt_text + prompt_text
        elif quadrupled:
            prompt_text += prompt_text
            prompt_text += prompt_text

        conversation = [
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": prompt_text},
                    {"type": "image"},
                ],
```

```
            },
        ]
        record = {
            "id": question["question_id"],
            "image": Image.open(os.path.join(IMAGE_PATH, question["
                image"])),
            "prompt": processor.apply_chat_template(conversation,
                add_generation_prompt=True),
            "label": question["label"],
        }
        record["image"].load()
        inputs.append(record)

    if max_num_questions > 0:
        inputs = inputs[0:min(max_num_questions, len(inputs))]

    return inputs


def evaluate_one(model, question, benchmark, processor, accumulator):
    """Performs one question/answer cycle."""
    response = configs.query_model(model, benchmark, processor,
        question["prompt"], question["image"])
    #print(response)

    next_word_regex = r"ASSISTANT:?[\s\"]*(\w+)"
    VALID_WORDS = set(["yes", "no"])
    accumulator.n += 1
    try:
        match = re.search(next_word_regex, response, re.IGNORECASE)
        word = match.group(1).lower()
    except Exception as e:
        accumulator.errors.append([question, response + ':' + str(e)])
    else:
        if word not in VALID_WORDS:
            accumulator.errors.append([question, response])
        else:
            if word == "yes":
                if question["label"] == "yes":
                    accumulator.tp += 1
                else:
                    accumulator.fp += 1
            else:
                if question["label"] == "no":
                    accumulator.tn += 1
                else:
                    accumulator.fn += 1
    #print(str(accumulator))


def perform_benchmark(model_name, benchmark_name, pope_split,
    max_num_questions, prompt_suffix):
    processor = configs.load_processor(model_name)
    model = configs.load_model(model_name)
    benchmark = configs.load_benchmark(benchmark_name, processor)
    questions = load_questions(
        f"/home/ncjones/code/VCD/experiments/data/POPE/coco/coco_pope_
            {pope_split}.json",
        processor,
        max_num_questions,
        prompt_suffix)
    accumulator = Accumulator()
    for question in tqdm(questions):
        evaluate_one(model, question, benchmark, processor,
            accumulator)
```

```
    return accumulator


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--benchmark", type=str, default="baseline")
    parser.add_argument("--pope_split", type=str, default="random")
    parser.add_argument("--prompt_suffix", type=str, default="default")
    parser.add_argument("--seed", type=int, default=55)
    parser.add_argument("--limit", type=int, default=-1)  # Maximum number of questions.
    parser.add_argument("--model", type=str, default="llava-7b")
    parser.add_argument("--runs", type=int, default=1)
    args = parser.parse_args()
    leading_header = 'id'
    leading_vals = ''
    for (col, val) in vars(args).items():
        leading_header += ',' + col
        leading_vals += ',' + str(val)


    if args.seed > 0:
        logging.warning(f"Setting seed to {args.seed}")
        set_seed(args.seed)
    results = []
    # This includes all the info from the argparser so that we can just combine
    # everything and do stats in a simple script.
    print(leading_header + ",TP,TN,FP,FN,Acc,Errors")
    for i in range(args.runs):
        result = perform_benchmark(
            args.model,
            args.benchmark,
            args.pope_split,
            args.limit,
            args.prompt_suffix)
        print(str(i) + leading_vals + "," + str(result))
```

This file performs the same thing, but in a query interface so you can analyze your own clown photos and ask random questions.

Listing 5: run_query.py

```
"""This is a driver program for running a prompt through a benchmark.
    """

import argparse
import configs
import benchmark
import torch
import logging

from PIL import Image
from transformers import set_seed


def execute_query(model_name, benchmark_name, query, image, max_new_tokens=4000):
    """Performs one question/answer cycle."""
    processor = configs.load_processor(model_name)
    model = configs.load_model(model_name)
    benchmark = configs.load_benchmark(benchmark_name, processor)

    conversation = [
```

```
        {
            "role": "user",
            "content": [
                {"type": "text", "text": query},
                {"type": "image"},
            ],
        },
    ]
    prompt = processor.apply_chat_template(conversation,
        add_generation_prompt=True)

    response = configs.query_model(model, benchmark, processor, prompt
        , image, max_new_tokens=max_new_tokens)
    print(response)


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--benchmark", type=str, default="baseline")
    parser.add_argument("--seed", type=int, default=55)
    parser.add_argument("--model", type=str, default="llava-7b")
    parser.add_argument("--image", type=str, default=None)
    parser.add_argument("--query", type=str, default="Please describe
        this image.")
    args = parser.parse_args()
    if args.seed > 0:
        set_seed(args.seed)
    execute_query(args.model, args.benchmark, args.query, Image.open(
        args.image))
```

6.4 Data analysis

This file collects and generates Table 4.

Listing 6: gather_results.py

```
import pandas as pd
import sys


def process(args):
    # Read the files.
    df = pd.concat(
        (pd.read_csv(f, header=0, usecols=[
            'benchmark',
            'pope_split',
            'model',
            'TP',
            'TN',
            'FP',
            'FN',
            'Acc'
        ]) for f in args[1:]), ignore_index=True)
    # Compute prec, rec, f1
    df['Precision'] = 100 * df['TP'] / (df['TP'] + df['FP'])
    df['Recall'] = 100 * df['TP'] / (df['TP'] + df['FN'])
    df['F1'] = 100 * (2 * df['TP']) / (2 * df['TP'] + df['FP'] + df['
        FN'])
    df['Acc'] = df['Acc'].str.replace('%', '', regex=False).astype(
        float)
    df.drop(labels=['TP', 'TN', 'FP', 'FN'], axis=1, inplace=True)

    # Compute summary statistics
    res = df.groupby(by=['pope_split', 'model', 'benchmark'])
```

```
    print('Means')
    print(res.mean().to_string(float_format="{:4.2f}".format))
    print('----')
    print('Standard deviations')
    print(res.std().to_string(float_format="{:4.2f}".format))


if __name__ == '__main__':
    process(sys.argv)
```

This file performs the benchmark analysis.

Listing 7: gather_baseline_thresholds.py

```
"""
The input data here consists of n runs of m different Semantically
    Equivalent
prompts, with POPE accuracy values computed (in theory, we could do
    this
with prec, rec, and F1, but let's just stick with Accuracy because it
    should
be good enough).  One of those m prompts is the "default" one, which
    is what
is used in the original VCD paper.

We need to take each of the reminaing $m-1$ prompts and perform a
one-sided (greater) Welch t-test; we use Welch here because we don't
    want
to assume equal variances, which is what Student's t-test does. From
    the t-test
we obtain Welch's degrees of freedom (which we could also compute by
    hand
but someone already wrote that in scipy).

We also compute the standard error of the delta between methods, which
     is
$S_i = \sqrt{s_0^2/n+s_i^2/n}$.

We then compute the critical value of the t-distribution for alpha
    =0.05 and the
df, which we call t_crit.  Now, $t_crit*SE_i$ gives us a threshold
    $T_i$ on the
delta between baseline decoding and some method's treatment of
    decoding, for prompt
method $i$; taking $T = max_i \in m T_i$ among the $m$ different
    methods gives
us an empirical bound on what we should expect to see, before
    concluding that
a decoding treatment truly is better than the baseline.

In our case, $m=11$ and $n=10$.
"""
import pandas as pd
import numpy as np
from scipy.stats import ttest_ind, t
import sys


# Quantile choice for "significance"
ALPHA = 0.05


def _extract(d, s):
    tmp = d[d['prompt_suffix'] == s].reset_index()
```

```
    # For the moment, let's see what happens if we only use 5 observations instead of 10.
    return tmp[:5]


def process(args):
    # Read the files.
    df = pd.concat(
        (pd.read_csv(f, header=0, usecols=[
            'prompt_suffix',
            'Acc'
        ]) for f in args[1:]), ignore_index=True)
    # Compute prec, rec, f1
    df['Acc'] = df['Acc'].str.replace('%', '', regex=False).astype(float)
    methods = [x for x in df['prompt_suffix'].unique() if x != 'default']
    df_base = _extract(df, 'default')
    var_base = df_base['Acc'].var()
    n_base = df_base['Acc'].count()
    mean_base = df_base['Acc'].mean()
    print(f'Baseline & ${n_base}$ & ${mean_base} (\\pm {np.sqrt(var_base):.2f}$')
    assert n_base > 0, 'There are no observations for baseline'
    Ts = []
    for m in methods:
        df_m = _extract(df, m)
        mean_m = df_m['Acc'].mean()
        var_m = df_m['Acc'].var()
        n_m = df_m['Acc'].count()
        assert n_m > 0, f'There are no observations for method {m}'
        se_m = np.sqrt(var_base/n_base + var_m/n_m)
        test_res = ttest_ind(df_m['Acc'], df_base['Acc'], equal_var=False, alternative='greater')
        dof_m = test_res.df
        t_crit_m = t.isf(ALPHA, dof_m)
        T_m = t_crit_m * se_m
        print(f'{m} & ${n_m}$ & ${mean_m} (\\pm {np.sqrt(var_m):.2f})$ & ${T_m:.2f}$') # (Dof={dof_m:.2f}, SE_m={se_m:.2f}, t_crit={t_crit_m:.2f})')
        Ts.append(T_m)

    print(f'Based on the provided data, the threshold for concluding improvement over baseline in POPE is T={np.max(Ts):.2f}')


if __name__ == '__main__':
    process(sys.argv)
```

There are also shell scripts to run the above programs, but these are omitted.